Unified Ransomware Detection & Recovery System

Phase 1-4 Team Explainer — How Every Part Works

Purpose: a walkthrough of the whole system, role by role (AS, NI, SI, SH), so every team member can explain their own part and understand everyone else's before the presentation. This is written from the code as it exists today, not from the original plan — where the two differ, the code is described.

Scope: Phase 1-4, Weeks 1-16 (Semester 1). Reference: *Unified Ransomware Detection & Recovery System — Complete Project Documentation*, v1.6.

---

0. The System in One Page

The project is five backend microservices, a dashboard, and an API gateway, all talking to each other over HTTP inside Docker containers. Nothing is a single monolith — each role owns one or two independently runnable, independently tested services.

Service	Port	Owner role	Job in one line
API Gateway	8000	SH	The only door in. Authenticates, rate-limits, forwards.
Monitor	8001	AS	Watches the filesystem, scores entropy, decides suspicious or not.
ML Engine	8002	NI	Runs the trained models, returns a prediction and confidence.
Ledger	8003	SI	Append-only hash-chained audit log — the "blockchain" layer.
Response	8004	AS + SI	Kills the attacking process, isolates the host, and restores files from backup.
Dashboard	8501	SH	Streamlit UI showing everything above, live.

The one sentence that matters most: a file gets written → the Monitor measures its entropy and byte structure → if that looks like encryption, the ML Engine scores it, the Ledger records it permanently, and the Response service kills the process and can restore the file from a Windows VSS snapshot. Every one of those four steps is a separate, independently owned microservice, glued together by the Gateway and visualised on the Dashboard.

0.1 The two flows that matter

Benign file write (nothing suspicious): Monitor reads the file once, computes entropy and a magic-byte check, decides it's fine, and just logs the event in memory for the dashboard. Nothing else is called — no ML, no ledger, no response. This is deliberate: it is what keeps the system's CPU usage near 1% instead of scoring every save of every document.

Ransomware-shaped file write (looks like encryption): Monitor flags it suspicious in under 100ms, then hands off to a background worker thread that fans out to three services in sequence — ML Engine for a model opinion, Ledger to record the event permanently, and (if the combined threat level is high or critical) Response to kill the process and isolate the host. Each of those three hops is independent and best-effort: if the Ledger is briefly down, the process still gets killed; if the kill fails, it's still logged.

---

1. Role AS — Endpoint Detection & Response

Owns: services/monitor/ (the watcher and the entropy/classification logic) and half of services/response/ (the process-kill and network-isolation actions). This is the "front line" — the part that actually notices an attack happening and reacts to it in real time.

1.1 The core idea: entropy, not signatures

AS's detector does not look for known ransomware families or virus signatures. It looks at **what encryption does to a file's bytes**, which is true of every ransomware family, known or brand new (this is also how it satisfies the "zero-day detection" test case, TC-02 — there is no signature to match against, so detecting an unseen sample proves the approach works).

The key measurement is **Shannon entropy**, a number from 0 to 8 that says how random a stream of bytes looks:

- A plain text file: entropy around 4–5 bits/byte (letters repeat a lot, so it's predictable).
- A photo or a ZIP file: entropy around 7.2–7.9 (already compressed, so it looks fairly random — this is the tricky case, see below).
- Encrypted data: entropy at or above 7.5–8.0, indistinguishable from random noise. This is mathematically unavoidable — strong encryption is defined by turning data into something statistically indistinguishable from random bytes.

The exact formula used (services/monitor/detection.py, function calculate_entropy/_entropy_from) is the textbook Shannon entropy: for every byte value 0–255, count how often it appears in the first 1 MB of the file, turn that into a probability, and sum -p * log2(p) across all 256 values. A file using only one byte value scores 0.0 (completely predictable); a file where all 256 byte values are exactly equally likely scores the maximum, 8.0 (completely unpredictable). This was unit-tested against the three values that must always hold (uniform random = 8.0, a file of one repeated byte = 0.0, a file alternating between two byte values = 1.0) to make sure the formula itself is correct before trusting it against real files.

1.2 The problem with entropy alone, and how it's solved

A plain entropy threshold has an obvious flaw: a legitimate ZIP file or JPEG is also high-entropy, because compression already removed the redundancy. Flagging every ZIP as ransomware would make the false-positive rate unusable. AS's classifier solves this with a decision chain in detection.py::classify(), checked in this specific order:

1. **Differential entropy check first.** The Monitor keeps a short history of entropy readings per file path (EntropyHistory class). If a file's entropy *rises* by 2.0 bits/byte or more and lands at 7.0 or higher, that's flagged as suspected encryption *regardless of what the file claims to be*. This is checked before anything else specifically because it is the one signal an attacker cannot fake: a ransomware strain that pastes a fake ZIP header onto its ciphertext (to fool a naive magic-byte check) still can't hide the fact that *this exact file* just jumped from ordinary entropy to near-random entropy. This matters enough that the spec calls it out by name three times as "Differential Entropy Analysis" — it is one of AS's signature contributions.
2. **Magic-byte container check.** If there's no big entropy jump, the Monitor reads the first 16 bytes of the file and checks them against 20 known container signatures (ZIP, PDF, JPEG, PNG, Office Open XML, etc. — identify_container()). If the high entropy is *explained* by a real container header, it's treated as benign compression, not encryption. If it's high-entropy with **no** explaining header, that's suspected encryption.
3. **Unreadable files** (locked by another process, permission denied) are reported as a distinct third outcome, unreadable — deliberately not folded into "benign," because on Windows a locked file is common and silently calling it safe would hide real attacks in progress.

The known, documented blind spot of this approach: **intermittent/partial encryption** (used by real families like LockBit 3 and BlackCat, which only scramble the first quarter of a file to go faster). Because the entropy measurement is over the whole file, an attack that only touches 25% of the bytes barely moves the average (measured around 5.2 bits/byte, well under the 7.5 threshold). This is called out explicitly in the project's own test suite (test_partial_encryption_is_a_known_blind_spot) rather than hidden — catching it would need per-block rather than whole-file entropy, which was judged not worth the false-positive cost this phase (compressed sections inside an ordinary .docx would start tripping it).

1.3 How the watcher actually sees a file change

AS uses the watchdog Python library, which wraps the operating system's native file-change notification API (inotify on Linux, ReadDirectoryChangesW on Windows). One subtlety the team had to solve: Docker Desktop on Windows passes a bind-mounted host folder into its Linux VM as filesystem type 9p, which does not support inotify at all. build_observer() in app.py detects this by reading /proc/mounts and automatically falls back to a PollingObserver (which just re-scans the directory periodically) when native events aren't available — and reports which backend it picked and why on GET /monitor/status, so this isn't a silent behavior change.

Every file event (created, modified, moved/renamed, deleted) funnels into one function, handle_event(), which is timed from the moment it starts to the moment a verdict exists — this exact interval is what "detection latency" means in the benchmark table (measured 23.7ms p95, against a <100ms target). Inside that function: read the magic bytes, read up to 1MB of the file once (both entropy *and* the byte-distribution statistics the ML model needs are derived from that single read — reading twice was tried first and cost roughly double the latency for no benefit), record the reading in the entropy history for that path, and classify.

1.4 What happens after a "suspicious" verdict

The watcher thread does not do the follow-up work itself — it hands the event to a background worker thread (pipeline.py::run()) and returns immediately, so the filesystem watcher is never blocked waiting on a network call. That pipeline:

1. Calls the ML Engine's /predict with the measured features.
2. Logs the event to the Ledger (always, regardless of the ML verdict).
3. If the *combined* threat level (see below) is high or critical, calls the Response service's /response/trigger.

A real bug the team found and fixed here is worth understanding, because it's a good illustration of why the two detectors (rule-based Monitor + ML model) have to agree, not just the ML model alone. The behavioural model needs entropy very close to the theoretical maximum (~7.995) before it's confident, and small encrypted files (under ~40KB) don't reliably reach that through sampling noise on a partial read. Before the fix, a small file encrypted in place would be correctly flagged suspected_encryption by the Monitor's own rule, but the ML model's low confidence score would report threat_level: low, and the Response service only acts on high/critical — so nothing happened. The fix (pipeline.py::effective_threat_level()) makes the effective threat level the *higher* of the Monitor's own verdict and the ML score: the model refines the decision, it does not get to overrule a detection the rule-based check already made.

1.5 Response actions (the other half of AS's territory)

services/response/actions.py implements two real actions:

- **Process termination** (terminate_process()): sends SIGTERM first, waits a grace period, and escalates to SIGKILL if the process hasn't exited. Guards against being pointed at PID 0, PID 1, a negative PID, or its own process. Measured 125.6ms end-to-end, against a <2s target (TC-07).
- **Network isolation** (isolate_host()): builds a platform-specific firewall rule plan (iptables on Linux, pfctl on macOS, netsh on Windows) but only *applies* it if an explicit environment flag, RESPONSE_ISOLATION_ENABLED, is set — otherwise it reports what it *would* have done. This is a deliberate safety valve: nobody wants a live demo to actually firewall the presenter's laptop off the network.

1.6 What AS should be ready to explain live

- Why entropy, not signatures — and what that buys (zero-day detection, TC-02).
- The three-branch decision in classify(), in order, and why the differential check comes first.
- The four simulated ransomware families and what each one specifically tests (locker = classic rename+rewrite; silent = same rewrite with no extension signal, so entropy alone has to catch it; copycat = create-new-then-delete-old, a different event shape; partial = the known, admitted blind spot).
- Detection latency: 23.7ms p95, against <100ms required.

---

2. Role NI — Machine Learning

Owns: services/ml-engine/ (the model-serving API) and src/ (everything about fetching data, training, and evaluating the models). NI's job is: given a description of a file's features, output a prediction (ransomware or benign) with a confidence score.

2.1 Two separate models, not one

This is an important distinction to have straight for the presentation — there are genuinely two different classifiers, trained on two different kinds of data, for two different jobs:

Model	Input	Trained on	Used for
EMBER / XGBoost static model	2,381-value feature vector describing a Windows executable's structure (PE headers, imports, sections)	50,000 real Windows PE files, 25,000 malicious / 25,000 benign, from the public EMBER research dataset	Classifying an arbitrary executable file as malware or not, without running it
Behavioural classifier	7 features describing how a file changed — entropy, size, magic bytes, modification rate, container format, ransom-style extension, and one derived rate value	A built corpus of real documents plus the same documents AES-encrypted, so the model learns "what does an encryption event look like" directly from labelled before/after pairs	Scoring the live file events the Monitor reports — this is the model that actually runs during the demo pipeline

The one the dashboard and the live attack-detection pipeline actually calls is the **behavioural** model. The EMBER/XGBoost model exists to satisfy the spec's requirement for a static-analysis classifier and to prove the "train a real ML model on a real 50,000-sample malware dataset" deliverable — both are trained, both are evaluated, and their comparison (XGBoost vs. a Random Forest baseline) is one of the required results tables.

2.2 What "70 features" means

services/monitor/pe_features.py parses a Windows executable (PE file format) and extracts 70 numeric/structural features per file: header fields, per-section entropy, whether any section is both writable *and* executable (a classic malware red flag — legitimate code almost never needs this), which Windows API functions the file imports grouped by what they do (functions like CryptEncrypt, WriteFile, TerminateProcess are flagged as behaviourally interesting), and export/resource/directory presence. The spec asked for "50+ features" — 70 clears that with room. If parsing fails for any reason, the function returns the same 70 keys with zero values rather than a different shape, which matters because a model expects its input vector to always be the same length in the same order.

2.3 Training pipeline, step by step

1. src/fetch_ember_subset.py — downloads only the data shards needed to build a balanced 50,000-sample set (25k malicious / 25k benign), with resume/retry so a dropped connection doesn't restart the whole download.
2. src/train_ember_model.py — splits the data 70/15/15 (35,000 train / 7,500 validation / 7,500 test, stratified so the class balance is preserved in every split), trains an XGBoost gradient-boosted tree classifier with scale_pos_weight tuned for the class balance, and writes the trained model plus its metrics to disk.
3. src/train_behavioral_model.py — separately builds the "before/after encryption" corpus and trains the 7-feature behavioural model. Labels here come from *how the file was actually produced* (untouched vs. AES-encrypted), not from the entropy rule — deliberately, so the ML model is an independent check on the Monitor's heuristic rather than trained to just reproduce it.
4. src/train_baseline_comparison.py — trains a Random Forest on the identical data/split for a same-data, same-seed comparison against XGBoost (this fulfils the spec's required "ML underperforms → fallback model" risk mitigation, and the results table showing XGBoost beating Random Forest on every metric).
5. src/shap_analysis.py — runs SHAP (SHapley Additive exPlanations) over the trained EMBER model to produce a feature-importance report: which of the 2,381 inputs actually drove the model's decisions. This is offline analysis for documentation/explainability, not called during live prediction.

All training is **deterministic** — a fixed random_state=42 means re-running training on the same data produces a byte-identical model file (same SHA-256 hash) every time. This was actually used as forensic proof during the project's own internal audit: when the metrics file on disk looked stale, re-training and comparing the resulting model's hash, file size, and SHAP output byte-for-byte against the deployed model proved they were already the same model, and the discrepancy was in a stale report file, not the model itself.

2.4 Serving predictions: services/ml-engine/app.py

Both trained models are loaded once at service startup, not per-request. The /predict endpoint looks at the shape of the incoming payload to decide which model to use: a 2,381-number ember_vector routes to the static PE model, a features object with the 7 named behavioural fields routes to the behavioural model. If the model file failed to load (e.g. training hasn't been run yet), the service doesn't crash — it returns HTTP 503 with the exact training command needed to fix it.

Feature importance returned alongside a prediction (visible in the dashboard's "Signals Driving the Decision" chart) comes from XGBoost's built-in global gain metric, not a full per-prediction SHAP explanation — that distinction matters if asked, since per-prediction SHAP is a documented, deliberately deferred item.

One correctness detail worth knowing: features.py::features_to_vector() has to interpolate the 3 byte-distribution statistics for any caller that doesn't measure them directly. This interpolation used to be based on the (wrong) assumption that encrypted data is "unprintable" — but uniformly random bytes are actually about 37% printable ASCII by chance (95 of 256 possible byte values fall in the printable range), so that assumption silently inverted the feature and was a real, fixed bug (documented as Finding F-2 in the completion summary).

2.5 Results, and what they mean

Model	Accuracy	Precision	Recall	F1	ROC-AUC
XGBoost (EMBER static PE)	95.77%	96.25%	95.25%	95.75%	99.23%
Random Forest baseline	93.89%	94.90%	92.77%	93.82%	98.59%
Behavioural classifier	88.43%	94.86%	81.27%	87.54%	95.85%

Explaining these terms simply, for the presentation:

- **Accuracy** — percent of all predictions that were correct.
- **Precision** — of everything the model called "ransomware," what fraction actually was. High precision means few false alarms.
- **Recall** — of everything that actually was ransomware, what fraction the model caught. High recall means few missed attacks.
- **F1** — the balance of precision and recall in one number.
- **ROC-AUC** — how well the model separates the two classes across every possible decision threshold, from 0.5 (no better than a coin flip) to 1.0 (perfect separation).

A separate, arguably more important number: tested specifically on 752 samples from real, named ransomware families present in the corpus (WannaCry, GandCrab, CryptoLocker, Cerber, Petya, Locky, TeslaCrypt), the system achieves **99.87% recall and 100% precision** — i.e. on real ransomware families specifically, essentially nothing gets missed and there were no false alarms in that set.

2.6 What NI should be ready to explain live

- Two models, two jobs: static PE analysis (EMBER/XGBoost) vs. live behavioural scoring (7-feature model) — and which one the demo pipeline actually calls.
- Why XGBoost was chosen over Random Forest (it wins on every metric on identical data), and that Random Forest was trained anyway, as the spec's own risk-mitigation fallback.
- What the 70 PE features roughly are (header/section/import structure), and why a fixed 70-key output even on parse failure matters (input shape stability).
- Precision/recall/F1/ROC-AUC in plain language, and the headline numbers.

---

3. Role SI — Blockchain-Style Ledger & Recovery

Owns: services/ledger/ (the tamper-evident audit trail) and half of services/response/ — specifically recovery/vss_manager.py and recovery/recovery.py (Windows backup snapshots and file restoration). SI answers two questions: "can we prove nothing in the audit trail was tampered with?" and "can we actually get the encrypted files back?"

3.1 The hash chain: a blockchain concept without an actual blockchain

The spec calls for "blockchain-style" tamper evidence. Rather than a real distributed blockchain (which needs a network of nodes and, for something like Polygon, real transaction fees — explicitly scoped to a later semester, see 3.4 below),

SI implemented the core idea a blockchain relies on — a **hash chain** — using nothing more than SQLite and Python's built-in hashlib.

The idea, concretely (services/ledger/hash_chain.py): every event that gets logged becomes a "block." Each block stores a SHA-256 hash computed over **its own contents plus the previous block's hash**:

```
current_hash = SHA256(timestamp + event_type + event_data_json + previous_hash)
```

This one design choice is what makes the whole chain tamper-evident. If anyone edits a single character in any historical block's data, that block's stored hash no longer matches what you'd recompute from its (now-changed) contents — and because the *next* block's hash was computed using the original (now-wrong) hash as an input, every block after the tampered one becomes invalid too. The very first block (the "genesis" block) has no predecessor, so its previous_hash is defined as 64 zero characters. verify_chain() walks every block in order, from a completely fresh database connection (so it can't be fooled by cached/in-memory state), and reports the exact block ID where the chain first breaks, distinguishing two different kinds of tampering: a block whose stored hash doesn't match its own recomputed content, versus a block whose previous_hash pointer doesn't match the block that's actually before it (which is what a deleted or reordered row looks like).

One implementation detail that mattered in practice: canonical_json() always serialises the event data with sorted keys and no extra whitespace, so the exact same event data always produces the exact same bytes to hash — without this, the same logical event could hash differently depending on, say, dictionary key ordering, which would make verification unreliable for reasons that have nothing to do with actual tampering.

3.2 Why writes are single-file and locked

add_block() is the **only** function anywhere in the codebase allowed to write a new block. It takes a lock, reads the current last block ("the tip"), computes the new hash from it, inserts, and commits — all inside that lock. This matters under concurrency: TC-11 specifically tests multiple simultaneous attacks being detected and logged at once, and without the lock, two concurrent writes could both read the same "tip" and each compute a hash chained off it, silently forking the chain into two inconsistent branches. The database itself is also configured deliberately conservatively (journal_mode=DELETE, not the more common WAL mode, and synchronous=FULL) — slower, but the safer choice for an audit log where "we might lose the last write on a crash" is not acceptable.

3.3 File recovery: Windows Volume Shadow Copy (VSS)

Detecting and stopping an attack only gets the victim halfway — files that were already encrypted before detection still need to come back. SI's recovery system uses Windows' built-in **Volume Shadow Copy Service (VSS)**, the same technology behind Windows' "Previous Versions" / File History feature, rather than building a custom backup system from scratch.

- **Automatic snapshots** (vss_manager.py::start_scheduler()): a background scheduler (APScheduler) takes a new VSS snapshot every 6 hours, so there's always a recent, complete point-in-time copy of the protected volume to restore from.
- **Snapshot creation** (create_snapshot()) calls into Windows' WMI (Win32_ShadowCopy.Create) API. Measured at 2.8 seconds, well under the 30-second target. One practical debugging note worth keeping: the underlying vssadmin tool writes its error messages to **standard output**, not standard error, which is the opposite of what most command-line tools do — the code specifically parses the right stream because of this.
- **Restoration** (recovery.py::RecoveryManager.restore_file()): given a damaged file's path, finds that same relative path inside the chosen snapshot and copies it back over the encrypted version.
- **Integrity verification**: after restoring, the recovered file's hash is compared against the *last known-good hash the Ledger recorded for that exact path* (via a small HTTP client, ledger_client.py) — tying SI's two areas together: the ledger isn't just a log, it's also the source of truth for "was this restore actually correct." If no prior hash exists for that path, the result is honestly reported as **unverified** rather than silently reported as a pass.

Recovery was measured at **100% file restoration success** (TC-04), and because VSS is only a real Windows feature, the code detects and reports when it's running somewhere VSS isn't available (e.g. inside a Linux container) via platform_status(), and falls back to a directory-backed stand-in for testing rather than pretending to succeed.

3.4 What's intentionally not built yet

The spec's Weeks 17–24 deliverable for SI is anchoring ledger hashes to the Polygon blockchain testnet (a real public blockchain) using Solidity smart contracts and web3.py. This is explicitly out of scope for Phase 1-4 — the hash chain described above is the "fallback" mitigation the project's own risk register names for exactly this case ("Blockchain costs → testnet, hash chain fallback"), and it is the mechanism actually shipped and tested this phase. blockchain_anchor is a real field on every ledger block already, always currently null, ready for that work to slot in later without a schema change.

3.5 What SI should be ready to explain live

- The hash-chain formula and, specifically, *why* chaining each hash to the previous one is what makes tampering detectable — changing history requires recomputing every hash after the change, and any observer holding the true chain can immediately tell it diverged.
- Live demo: scripts/si_demo.py deliberately tampers a row in the ledger and shows /ledger/verify catching it and

reporting the exact block, then restores the row so the demo is repeatable.
- VSS is a real, existing Windows feature being orchestrated, not a from-scratch backup system — and the 6-hour automatic snapshot schedule.
- Ledger verification measured at 3.1ms median over 1,000 blocks, against a <50ms target.

---

4. Role SH — Integration, Gateway, Dashboard & DevOps

Owns: services/gateway/ (the single authenticated entry point), services/dashboard/ (the Streamlit UI), and docker-compose.yml (how all six services actually get started and wired together). SH does not own any detection or ML logic — the job is making everyone else's service reachable, secure, observable, and one-command-deployable.

4.1 Why a gateway at all, instead of calling each service directly

Every backend service (Monitor, ML Engine, Ledger, Response) binds only to 127.0.0.1 (localhost) inside its container — they are not reachable from outside the Docker network at all. The Gateway is the **only** service that publishes a port to the outside world (8000, plus the Dashboard's 8501). This means every request from a real client, and every request the Dashboard makes, has to pass through one place that can enforce authentication and rate limits consistently, rather than every backend service re-implementing its own security independently (and inevitably inconsistently).

4.2 Request lifecycle through the gateway

Every incoming request passes through this exact chain (services/gateway/):

1. **Request-ID middleware** (main.py) — tags every request with a unique X-Request-ID for tracing.
2. **Authentication** (auth.py::get_current_user) — decodes and verifies the JWT (JSON Web Token) bearer token's signature. Fails with HTTP 401 if missing or invalid.
3. **Authorisation** (auth.py::require_role) — checks the token's role claim against what the specific endpoint requires (e.g. starting/stopping the Monitor requires admin). Fails with 403 if the role doesn't qualify — and on *any* 401 or 403, the gateway writes an auth_failure entry to the Ledger itself (this satisfies TC-10: "API authentication failure → 401 returned, audit log entry created" and is a nice, concrete example of the Gateway and the Ledger cooperating).
4. **Rate limiting** (rate_limit.py) — a token-bucket limiter keyed by the caller's subscription tier (Free: 60 req/min, 100 burst; Premium: 300/500; Enterprise: 1000/2000). Fails with 429 if exceeded.
5. **Proxying** (routers/proxy.py) — forwards the (now authenticated, authorised, rate-checked) request to the real backend service over the internal Docker network, and translates any downstream error into one consistent JSON error shape used everywhere in the system.

JWT, explained simply: a JWT is a signed, tamper-evident token that encodes who the caller is claimed to be. This project's tokens carry sub (subject/username), role, tier, iat (issued-at time) and exp (expiry time), all signed with a shared secret (JWT_SECRET) using HMAC-SHA256. Anyone holding the secret can mint a valid token; anyone without it cannot forge one, because changing any field invalidates the signature. In development, POST /auth/token can mint tokens directly for convenience, but minting anything above the lowest ("free") privilege level requires a separate bootstrap secret header — and the gateway explicitly refuses to even start up if the placeholder development secret is still in use while running in a production-flagged environment (verify_jwt_secret_configuration()).

4.3 The composite /analyze endpoint

One gateway endpoint, /analyze, doesn't just forward — it orchestrates a full three-step chain on the caller's behalf: call the Monitor's /features to extract a file's features, feed that straight into the ML Engine's /predict, then log the resulting prediction to the Ledger — and returns the final prediction. This is the gateway-level equivalent of the automatic attack pipeline described in Section 1.4, but triggerable on demand for any single file rather than only firing automatically on a live suspicious event. Each of the three steps has its own distinct, named error code if it fails (FEATURE_EXTRACTION_FAILED, PREDICTION_FAILED, LEDGER_LOG_FAILED), so a caller (or a debugging teammate) can tell exactly which stage broke.

4.4 The Dashboard

services/dashboard/app.py, built with Streamlit, auto-refreshing roughly every second. It never talks to the backend services directly — every single piece of data it shows comes through the same authenticated Gateway path everything else uses (it mints its own admin-level token internally using the shared JWT secret, the same mechanism described above, just automated). The panels, and what each one is actually pulling from:

Panel	Backed by
Status banner (Secure / Threat Detected)	Combines the Monitor's own verdict on the latest event with the ML model's prediction — shows the worse of the two, so a low-confidence model score can't hide a detection the rule-based Monitor already made

Panel	Backed by
End-to-End Pipeline strip	/health, /predict, latest ledger block, response status
Current Event / Model Decision	/monitor/events, /predict, feature importance
Entropy Trend chart	Recent events plotted against the 7.5 threshold line
Ledger Evidence	/ledger/entries — block ID, current/previous hash, tamper-proof flag
Service Health table	/health, per service
Model Quality	/model/metrics, read live from the training reports on disk — so retraining the model updates this without any dashboard code change

4.5 Docker Compose: the one-command deployment

docker-compose.yml defines all six services, a shared internal network, per-service healthchecks, and CPU/memory limits (2 CPUs / 2GB per service, matching the spec exactly). docker compose up -d --build builds every service's image and starts the whole stack in dependency order — using condition: service_healthy rather than just "started," so, for example, the Gateway doesn't come up and start accepting traffic until the Monitor, ML Engine, and Ledger have all actually reported themselves healthy, not merely launched. This is what made the earlier live demo work smoothly once Docker Desktop itself was running: one command, and every service's dependency ordering, health, and networking is handled automatically rather than requiring five separate manual startup steps in the right order.

4.6 What SH should be ready to explain live

- Why one gateway, and what "the only door in" means for security — every backend port is loopback-only.
- The five-stage request pipeline (request-ID → auth → role check → rate limit → proxy) and which HTTP status code each stage can produce.
- JWT in plain terms: a signed claim of identity, not encryption of the identity — anyone can read the claims, only the holder of the secret can produce a valid signature for new ones.
- What /analyze actually chains together, and why each of its three steps has its own distinct error code.
- API response time: 3.22ms p95 over 1,000 requests, against a <200ms target; dashboard update latency 10.5ms to queryable data.

---

5. How It All Fits Together — One Attack, Start to Finish

This is the sequence worth being able to narrate as a group, each person picking up their own part:

1. **(AS)** A file gets rewritten with high-entropy content and a ransomware-style extension. The watcher's callback fires within milliseconds.
2. **(AS)** handle_event() reads the magic bytes and up to 1MB of content in a single pass, computes entropy and byte statistics, checks the differential-entropy history for that path, and classifies it — verdict: suspected_encryption. This whole step is what's measured as detection latency (23.7ms p95).
3. **(AS → NI)** The event is handed to a background worker, which calls the ML Engine's /predict with the measured features.
4. **(NI)** The behavioural model (trained on real encrypted-vs-untouched document pairs) scores the feature vector and returns a prediction, confidence, and threat level.
5. **(AS)** The effective threat level becomes the higher of the Monitor's own verdict and the model's score, so a low-confidence model reading cannot suppress a detection the rule-based check already made.
6. **(AS → SI)** The event, with its prediction, is logged to the Ledger, becoming an immutable, hash-chained block.
7. **(AS)** If the effective threat level is high or critical, the Response service terminates the offending process (125.6ms measured) and can apply network isolation.
8. **(SI)** A subsequent recovery request restores the affected file from the most recent VSS snapshot and verifies its hash against the last known-good hash the Ledger recorded for that exact path.

9. **(SH)** Every one of the steps above was reachable only because it passed through the Gateway's authentication and rate limiting, and every result — the verdict, the prediction, the new ledger block, the response action — is visible on the Dashboard within about 440 milliseconds of the original file write, against a 1-second target.

6. Testing, Honestly

The project runs 371 automated tests (165 unit, 177 integration, 25 end-to-end), all passing, with 2 legitimate platform-specific skips. Split by who owns what:

Area	Tests	What they check
Gateway (SH)	70	Auth, rate limiting, contract parity with the OpenAPI spec (in both directions — every documented route exists, and every implemented route is documented)
Ledger (SI)	42	Hash-chain integrity, tamper detection, verification speed
Monitor (AS)	90	Detection logic, entropy edge cases, PE feature extraction, benchmarks, the simulator, concurrent-attack handling
ML Engine (NI)	19	Both model paths (static and behavioural), input validation
Response + Recovery (AS + SI)	84 + 2 skipped	Termination behaviour, isolation planning, VSS snapshot/restore, concurrent kills

All 10 of the spec's performance targets (Table 5.9) are met, most with significant headroom — e.g. detection latency 23.7ms against a 100ms budget, response time 125.6ms against a 2-second budget. All 11 in-scope test cases (Table 5.8) pass; the 12th, blockchain anchoring, is explicitly Weeks 17–24 by the spec's own schedule and is not attempted.

Known, documented gaps — worth mentioning proactively in the presentation rather than waiting to be asked, since it demonstrates the team audited its own work rather than just claiming success:

- No TLS encryption between the internal services yet (mitigated by every backend port being loopback-only, so nothing crosses an actual network boundary unauthenticated).
- Intermittent/partial-file encryption is a known, tested, and explicitly asserted blind spot (Section 1.2 above).
- Blockchain anchoring to Polygon, CI/CD via GitHub Actions, and a written research paper are all Weeks 17–32 by the spec's own timeline — not attempted this phase, not gaps in this phase's scope.

7. Quick Glossary

Shannon entropy	A measure from 0–8 (bits/byte) of how random a stream of bytes looks. Encryption pushes it toward 8; encryption is what the Monitor is really watching for.

Magic bytes	The first few bytes of a file that identify its real format (e.g. ZIP files always start with PK), independent of the file extension.
PE file	Portable Executable — the file format Windows .exe/.dll files use. NI's static model reads structural features out of this format.
XGBoost	A gradient-boosted decision tree algorithm — builds many small decision trees in sequence, each correcting the previous ones' mistakes.
SHAP	SHapley Additive exPlanations — a method for explaining which input features drove a specific model prediction.
Hash chain	A sequence of records where each one's hash depends on the previous one's hash, making any retroactive edit detectable.
SHA-256	A cryptographic hash function: turns any input into a fixed 256-bit fingerprint; changing even one bit of the input completely changes the output.
VSS (Volume Shadow Copy)	A built-in Windows feature for taking point-in-time snapshots of a disk volume without stopping the system, used here for automated backups.
JWT	JSON Web Token — a signed, self-contained credential proving who a request claims to be.
Rate limiting / token bucket	Restricting how many requests a client can make per minute, with a small "burst" allowance on top of the steady rate.
p95 latency	The value below which 95% of measurements fall — a standard way to report "how slow is the slow case," ignoring rare outliers.

Generated as a study/presentation aid from the project's own code and documentation (docs/FLOW.md, docs/APPROACH.md, docs/PHASE1-4_COMPLIANCE_AUDIT.md, docs/PHASE1-4_COMPLETION_SUMMARY.md) as of 13 August 2026.